default V computeIfAbsent(K key, Function<? super K,? extends V> mappingFunction)

Predefined method of Map interface added from JDK 1.8V as a default method

It is used to COMPUTE & INSERT the value in the Map by using given Function<K,V>, If key is not available in the Map collection.

Important Points related to computeIfAbsent() :

Key already has a value => Do nothing, return that existing value.[return type is V]

Key is new or null => Create a new value (using the Function) and put it in the map.

If the function gives null => Don't put anything in the Map.

```java
package com.ravi.hash_map;

import java.util.HashMap;

//V computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction)

public class HashMapDemo7
{
    public static void main(String[] args)
    {
          HashMap<String, Integer> map = new HashMap<>();
          map.computeIfAbsent("Apple", key -> key.length() + 1);
          map.computeIfAbsent("Apple", key -> 20);
          map.computeIfAbsent("Mango", key -> 50);
          map.computeIfAbsent("Grapes", key -> 100);
          map.computeIfAbsent(null, key -> 90);
          map.computeIfAbsent("Kiwi", key -> null);

          IO.println(map);
    }
}
```

---

default V merge(K key, V value, BiFunction<? super V,? super V,? extends V> remappingFunction)

It adds the entry in the map, if the key is new, otherwise it combines old and new key value with the given function. If function result is null then key will be removed. It accepts entrire Map and perform operation based on the given key.

Different Cases :

a) If the key is missing OR has null [Insert the given value]
  map.merge("A", 10, (oldVal, newVal) -> oldVal + newVal);
  "A" not present in the map → map = {A=10}

b) If the key already has a value
 [Combine old value and new value using the function]
 map.merge("A", 5, (oldVal, newVal) -> oldVal + newVal);
 "A" was 10 → 10 + 5= 15 → map = {A=15}

c) If the function gives null [Remove the key from the map]
  map.merge("A", 5, (oldVal, newVal) -> null);
  Removes "A" from the Map. [because key A" is already available in the Map]

d) If key is not present and Function returns null then key will be inserted
  map.merge("Kiwi", 25, (oldValue, newValue)-> null);
  Note : Kiwi is not present as a key then It will be inserted.

NOte : The conclusion is, if Function returns null and key is available then remove the key from the map on the other hand if Function returns null but key is new key
then insert the entry in the map.

```java
package com.ravi.hash_map;

import java.util.HashMap;
import java.util.Map;

/*default V merge
    (K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)  */

public class HashMapDemo8
{
  public static void main(String[] args)
  {
     HashMap<String,Integer> map = new HashMap<>();
     map.merge("Pencil", 5 , (oldValue, newValue)->  newValue);
     map.merge("Pencil", 4 , (oldValue, newValue)->  oldValue + newValue);
     map.merge("Pencil", 9 , (oldValue, newValue)->  oldValue + newValue);
     map.merge("Eraser", 35, (oldValue, newValue)-> oldValue + newValue);
     map.merge("Eraser", 15, (oldValue, newValue)-> null); //remove the key
     map.merge("Sharpner", 25, (oldValue, newValue)-> null); //insert in the map
     IO.println(map);

     IO.println("...........................");

     HashMap<String,Integer> studyMap = new HashMap<>();
     studyMap.computeIfAbsent("Pen", key -> 50);
     studyMap.merge("Pen", 30, (oldValue, newValue)-> oldValue - newValue);
     IO.println(studyMap);
  }


}
```

default V computeIfPresent(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)

It changes the value only if the key is already present in the map. It will not insert the entry in the Map. If Function returns null, the key is removed from the Map.

Different Cases :

a) If the key exists in the map and its value is not null [Update the value using the function]

b) If the key is missing OR key is null [Do nothing]

c) If the function returns null [Remove the key]

```java
package com.ravi.hash_map;

import java.util.HashMap;

//default V computeIfPresent
//(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)

public class HashMapDemo9
{
    public static void main(String[] args)
    {
        HashMap<String, Integer> map = new HashMap<>();

        map.put("Java", 10);
         map.put("Python", 9);
         map.put("React", 5);
         map.put("HTML", 7);

         map.computeIfPresent("Java", (key, value) -> value + 5);
         map.computeIfPresent("CSS", (key, value) -> value + 5);
         map.computeIfPresent("HTML", (k,v) -> k.length() * v);
         map.computeIfPresent("React", (k,v) -> null);
         map.computeIfPresent(null, (k,v) -> k.length());


         IO.println(map);
     }

}
```

***Map with List of values [Map<K, List<V>>] :

```java
package com.ravi.hash_map;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

//Program that describes how many student has enrolled in a particular course
//Java ,[Raj, Samir, Ravi]

public class HashMapDemo10
{
    public static void main(String[] args)
    {
        HashMap<String, List<String>> courseMap = new HashMap<>();

        courseMap.computeIfAbsent("Java", key -> new ArrayList<String>()).add("Raj");
        courseMap.computeIfAbsent("Java", key -> new ArrayList<String>()).add("Samir");

        courseMap.computeIfAbsent("REACT", key -> new ArrayList<String>()).add("Scott");
        courseMap.computeIfAbsent("REACT", key -> new ArrayList<String>()).add("Alen");



        //Add students in the existing course by using computeIfPresent()
        courseMap.computeIfPresent("Java", (key, value) ->
        {
          value.add("Ravi");
            return value;
        });


        courseMap.computeIfPresent("REACT", (k,v)->
        {
        v.add("John");
        return v;
        });

        IO.println(courseMap);

    }
}
```

```java
package com.ravi.hash_map;

import java.util.Collections;
import java.util.HashMap;
import java.util.Map;

//cheapest flight & expensive flight

public class HashMapDemo11
{
  public static void main(String[] args)
  {
      Map<String, Integer> flights = new HashMap<>();
       flights.put("Mumbai", 5500);
       flights.put("Bangalore", 7000);
       flights.put("Chennai", 6500);
       flights.put("Vizag", 4500);
       flights.put("Nagpur", 4500);
       flights.put("Kolkata", 7000);


       //cheapest flight
       Integer min = Collections.min(flights.values());

       for(Map.Entry<String, Integer> entry : flights.entrySet())
       {
          if(entry.getValue().equals(min))
          {
              IO.println("Cheapest flight from Hyderabad to "+entry.getKey()+", Price is
:"+entry.getValue());

          }

       }


     //cheapest flight
       Integer max = Collections.max(flights.values());

       for(Map.Entry<String, Integer> entry : flights.entrySet())
       {
          if(entry.getValue().equals(max))
          {
              IO.println("Expensive flight from Hyderabad to "+entry.getKey()+", Price is
:"+entry.getValue());

          }

       }






   }
}
```